



Codeus practice

Outbox + Inbox pattern



About me

- Name: Bohdan Tkachuk
- Software Engineer @ MK-Consulting



Outbox pattern



Outbox Pattern

Ensures reliable event delivery in distributed systems

Guarantees atomicity and consistency between database and message queue



Outbox Pattern – Solved Problems

Data inconsistency between database and message broker

Example: service crash after DB commit but before event publish -> lost messages

Ensuring messages are eventually delivered even after system failures

Atomicity between business operation and event emission



Outbox Pattern - How It Works

1. Begin transaction

Start a database transaction to group all operations atomically

2. Write business data

Insert or update main entity data (e.g., create a new order)

3. Append outbox record (same TX)

Insert an event record into the outbox table within the same transaction

4. Commit once

Commit the transaction — both orders and outbox records are persisted together

5. Publisher picks up NEW events

A background worker (poller or CDC tool like Debezium) periodically reads outbox WHERE status='NEW'.

6. Publish to broker

Send the event payload to a message broker (e.g., Kafka topic order-created), usually including eventId in headers

7. Mark as sent / advance status

On successful publish, update record **status** to **SENT** and set **processed_at=NOW()**

8. Retries & observability

If publishing fails — leave status as NEW or mark FAILED; retry later and monitor lag/metrics for issues.



Outbox Pattern – Use Cases

Reliable Microservice Communication

Guarantee message delivery between services (e.g., Order → Payment) even if a service crashes after committing the transaction

Transactional Messaging in Financial Systems

Ensure that critical operations like payments, invoices, or refunds trigger corresponding events exactly once

Saga Pattern Orchestration

Coordinate multi-step distributed workflows (e.g., flight + hotel + payment booking) with guaranteed event delivery at each step

CQRS (Command Query Responsibility Segregation) Implementation

Use the Outbox pattern to propagate domain events from the write model to the read model asynchronously, ensuring consistency without coupling

Auditing and Event Replay

Maintain an auditable event trail in the **outbox** table, enabling replay for debugging, analytics, or recovery after failures



Outbox Poller Options

Custom Poller (Manual Implementation)

- Application-level scheduled job (e.g., Spring @Scheduled, Quartz, Cron).
- Periodically queries DB: `SELECT * FROM outbox WHERE status = 'NEW' LIMIT ...`
- Publishes events to broker (Kafka, RabbitMQ). Updates status to SENT or FAILED.

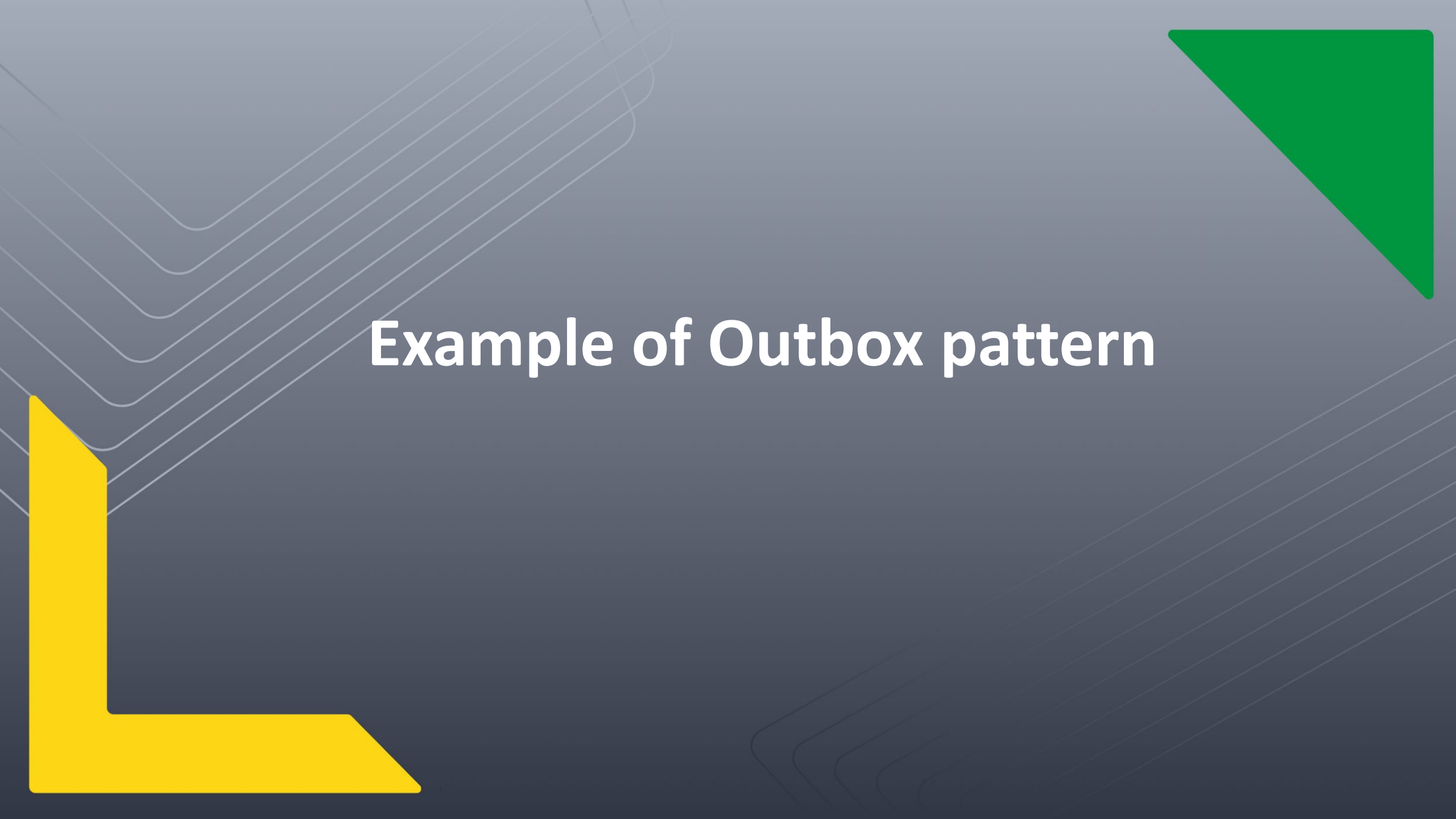
Debezium (CDC - Change Data Capture)

- Outbox table changes captured via Debezium (Kafka Connect).
- Debezium reads DB transaction log (binlog / WAL) → automatically publishes events to Kafka.
- Follows Outbox Event Router pattern (standard connector).

Stream-Based / Integration Tool

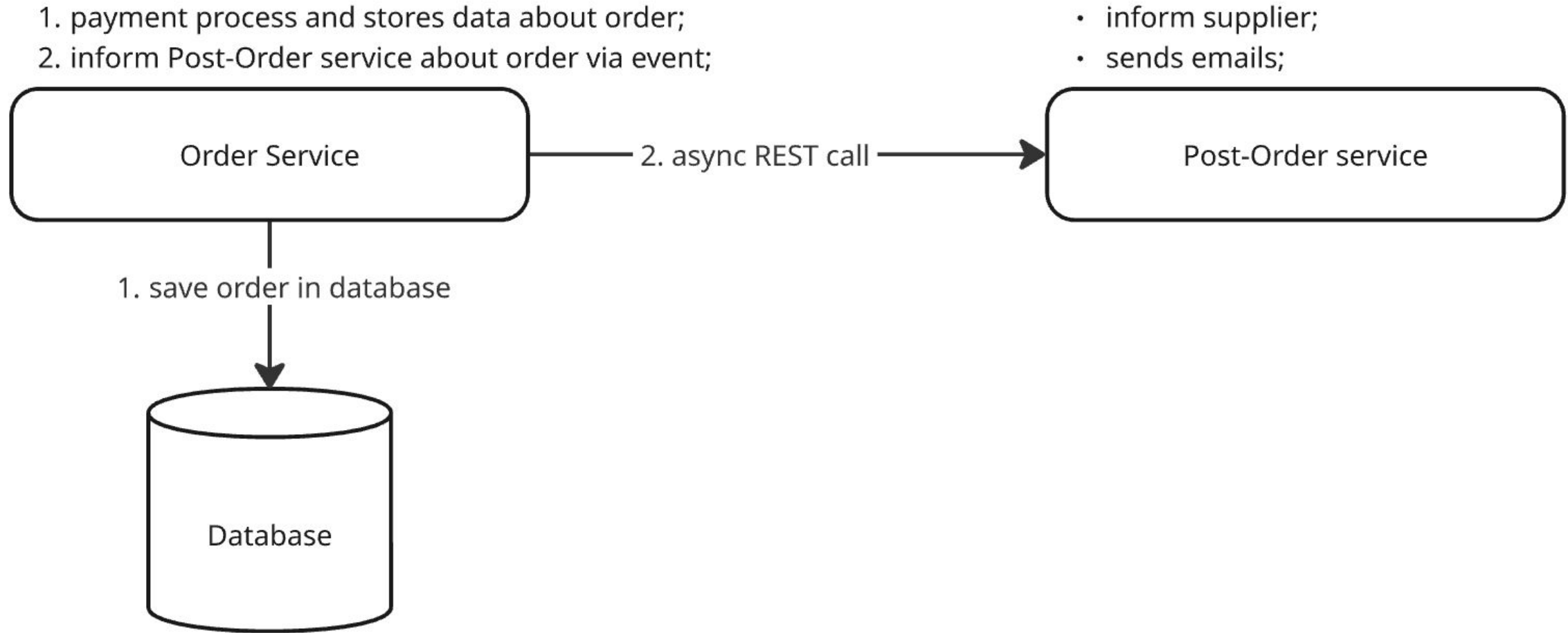
- Tools like Kafka Connect, AWS DMS, or Change Stream (MongoDB) can act as outbox pollers.
- Event streaming frameworks (e.g., Spring Cloud Stream, Flink) can subscribe to DB change topics.





Example of Outbox pattern

Architecture via REST



Architecture via REST

Props:

Simple to implement and debug

Immediate response, easy error handling

No need for extra infrastructure

Cons:

Tight coupling

OrderService depends directly on PostOrderService availability

No resilience

If PostOrderService is down, the request fails and order isn't processed

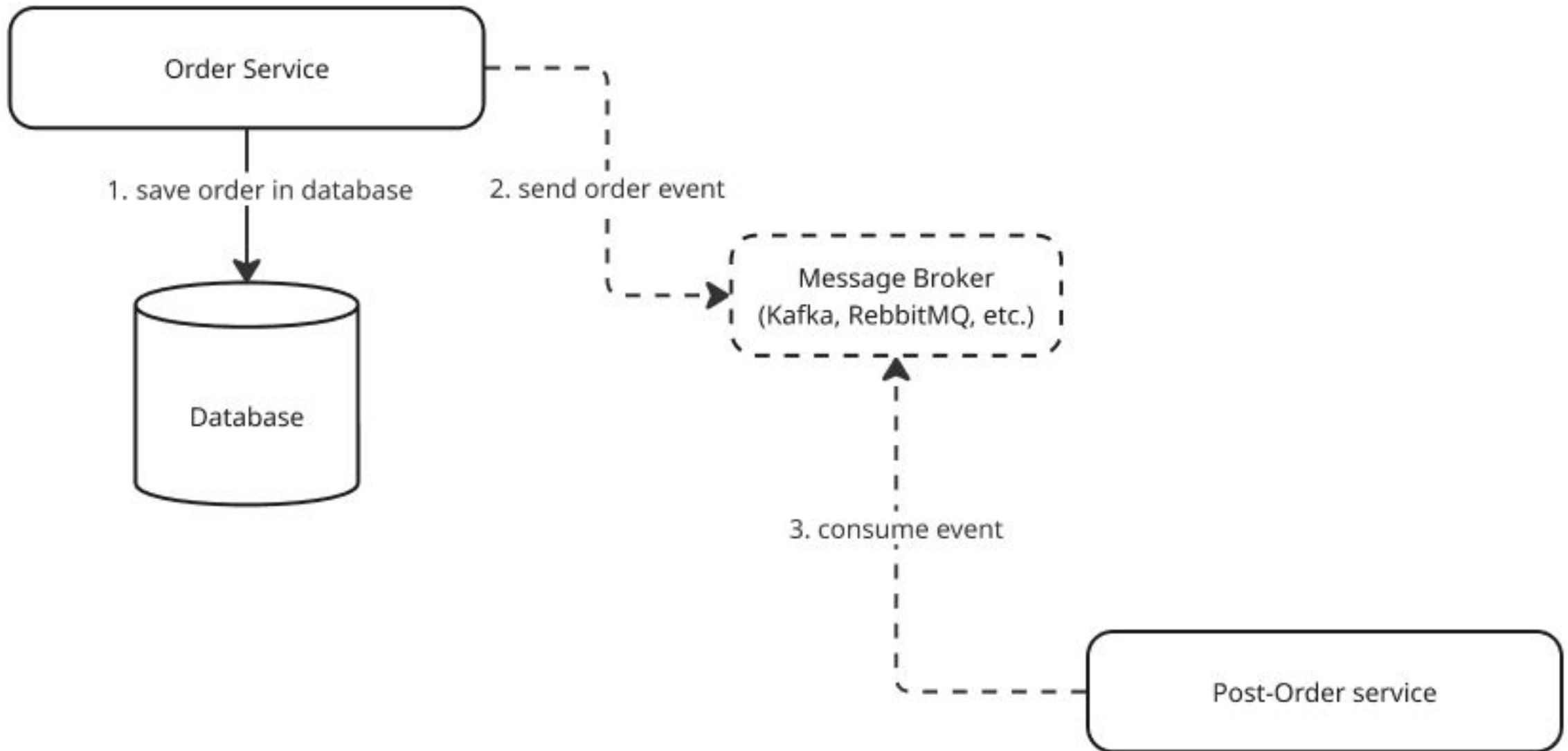
synchronous blocking call for each order

(optional) No event-driven flow

Other services can't easily react to order creation



Architecture via Kafka



Architecture via Kafka

Props:

Asynchronous and decoupled communication

Scales better under load

Multiple services can consume the same event

Cons:

Possible data inconsistency

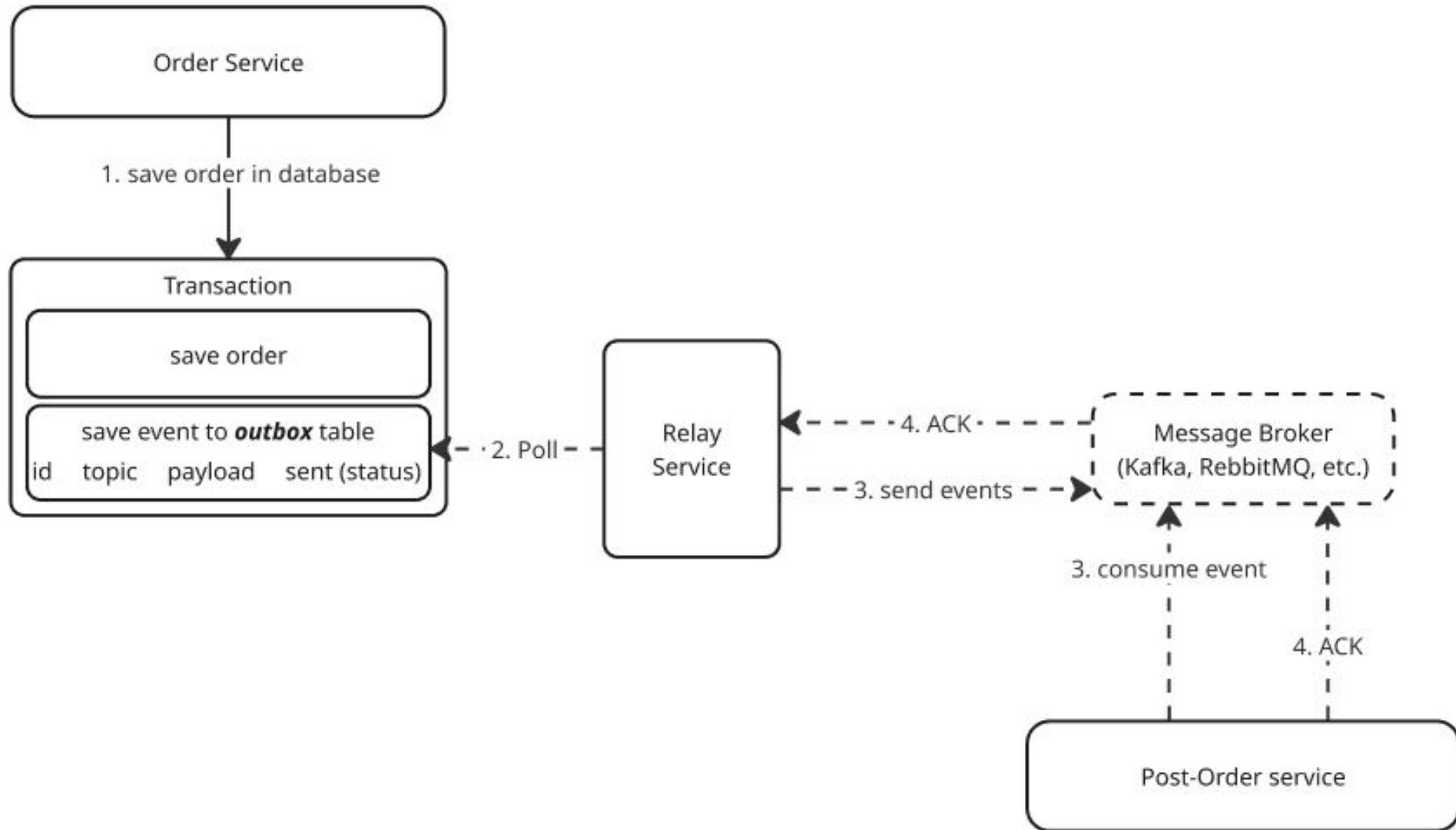
If OrderService crashes after saving order but before publishing Kafka event

No transactional guarantee between database and message broker

Lost messages in failure scenarios



Architecture via Kafka + Outbox pattern



Architecture via Kafka + Outbox pattern

Props:

Atomicity between business transaction and event publishing

Reliable delivery — no lost messages

Easy to replay or audit events from outbox table

Cons:

Event duplication possible

Poller might resend already processed events

Consumers may handle same message multiple times

Producer guarantees delivery, but consumer side lacks idempotency



Inbox pattern

Inbox Pattern

Ensures idempotent message processing in distributed systems

Prevents duplicate handling of the same event



Inbox Pattern – Solved Problems

Processing the same event multiple times → data corruption or double actions

Example: duplicate message delivery from brokers (at-least-once semantics)

Tracking which events were already processed

Ensuring message handling remains consistent during retries or failures



Inbox Pattern – Use Cases

Idempotent event consumption in microservices

Ensure each incoming event is processed exactly once, even if it's delivered multiple times

CQRS (Command Query Responsibility Segregation) Implementation

Guarantee that updates to read models from event streams happen only once for each event

Auditing processed messages

Record handled messages in the *inbox* table for traceability, troubleshooting, and compliance



Inbox Pattern - How it works

1. Consume event from broker

2. Begin transaction

3. Check inbox for eventId

SELECT 1 FROM inbox WHERE id = :eventId FOR UPDATE (or unique-key check)

4.1. If already exists → skip

Event was processed before → ack broker (or just commit) and return

4.2.1. Insert inbox record

```
INSERT INTO inbox(id, event_type, payload, received_at, status='RECEIVED')
```

4.2.2. Apply business action

Update local state/DB: e.g., create post-order record, send notification, etc.

7. Mark inbox as processed

```
UPDATE inbox SET status='PROCESSED', processed_at=NOW()  
WHERE id=:eventId
```

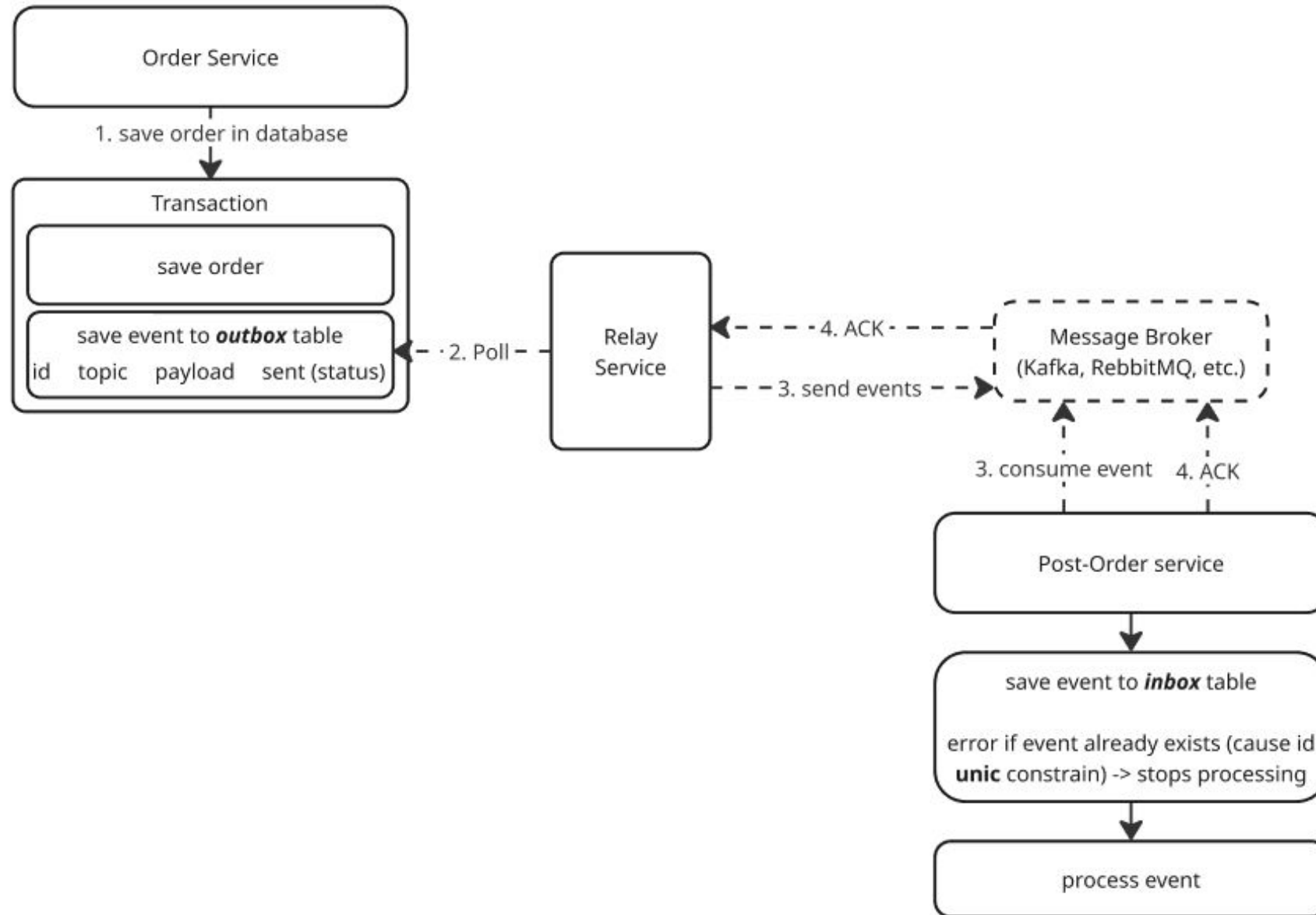
8. Commit & ack

Commit transaction; acknowledge message to broker



Example of Inbox pattern

Architecture via Kafka + Outbox & Inbox pattern



Architecture via Kafka + Outbox & Inbox pattern

Props:

End-to-end reliability — no lost, no duplicated events

Idempotent processing on consumer side

Strong consistency across distributed systems

Enables exactly-once semantics at business level

Cons:

More complex setup (extra tables, jobs, monitoring)

Slight delay in delivery due to async polling/CDC

Higher operational overhead



Thank you

- Author: Bohdan Tkachuk
- My LinkedIn: [link](#)
- Date: October 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

